

Lecture 12

ASIC Backend Design





Outline

- Basics of Backend Design (***)
- Cadence Innovus Tutorial (***)





ASIC Design Flow

- Application-specific Integrated Circuit (ASIC) Design Flow for Large-scale Digital Circuits

Design Language and Tools:

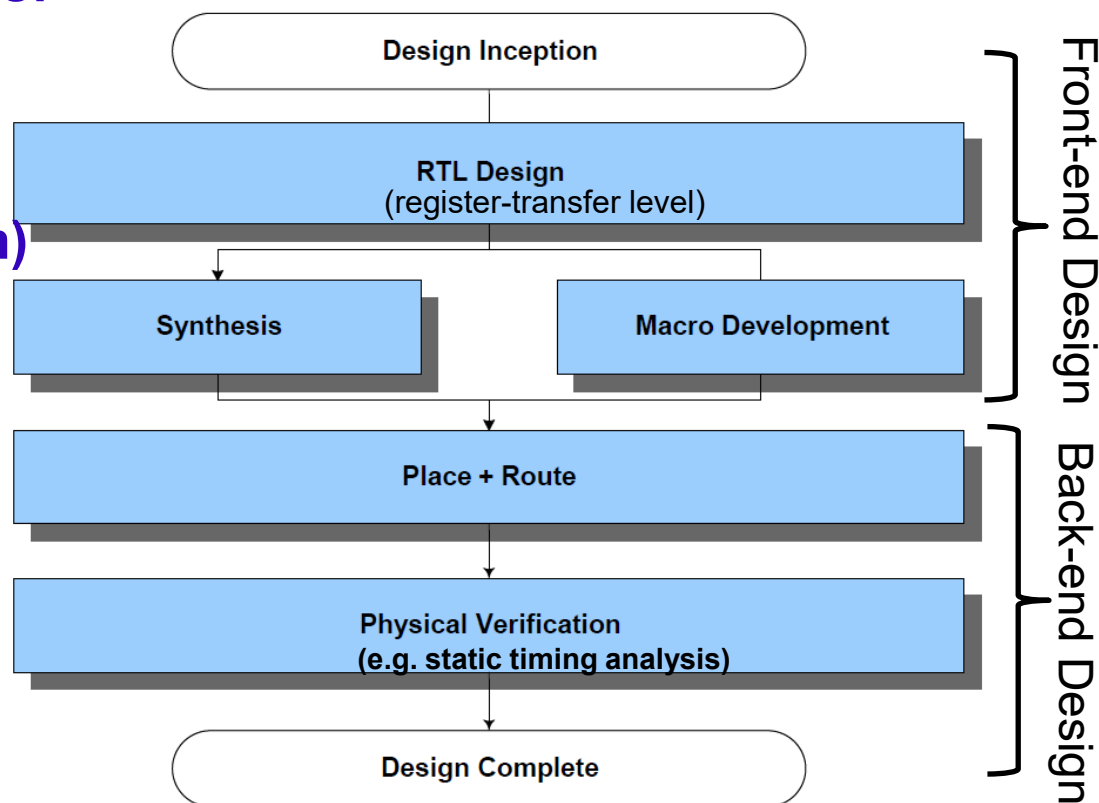
Matlab / C / Python →

Verilog/VHDL
(sim by Modelsim or Xcelium) →

Design Compiler
Or Genus →

Innovus →

Innovus or
Primetime →





Layout of Logic Gates

- Drawing that map circuit into physical design for fabrication
 - Referred as backend design or physical design
- Many layers of drawing:
 - Metal: 10+ layers
 - M1, M2, ... , M10
 - Transistors

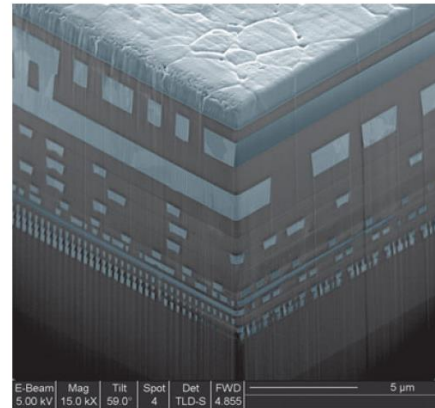


FIGURE 3.13 Cross-section showing 11 levels of metallization (Courtesy of International Business Machines Corporation. Unauthorized use not permitted.)

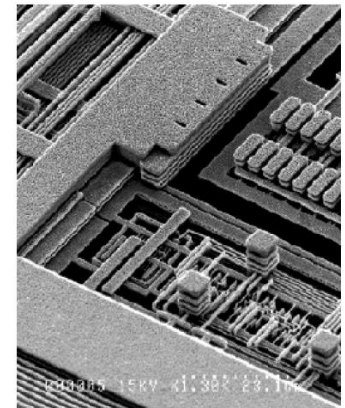


FIGURE 3.14 Micrograph of metallization in six-layer copper process (Courtesy of International Business Machines Corporation. Unauthorized use not permitted.)



Mask and Layout

E-beam Lithography to write masks (extremely slow and expensive)

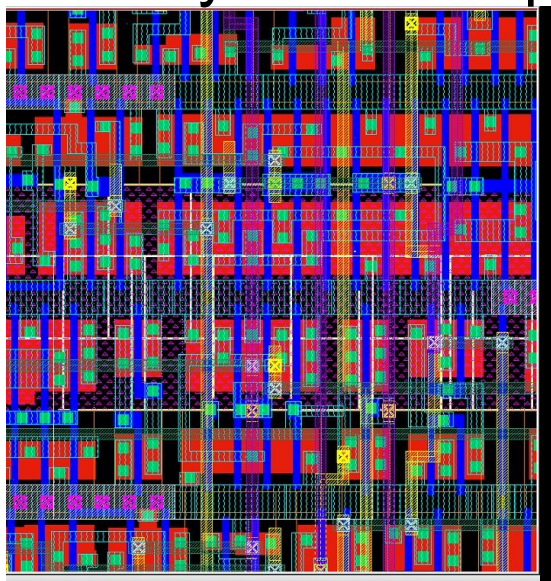
28 masks -> 180nm

49 masks -> 32nm

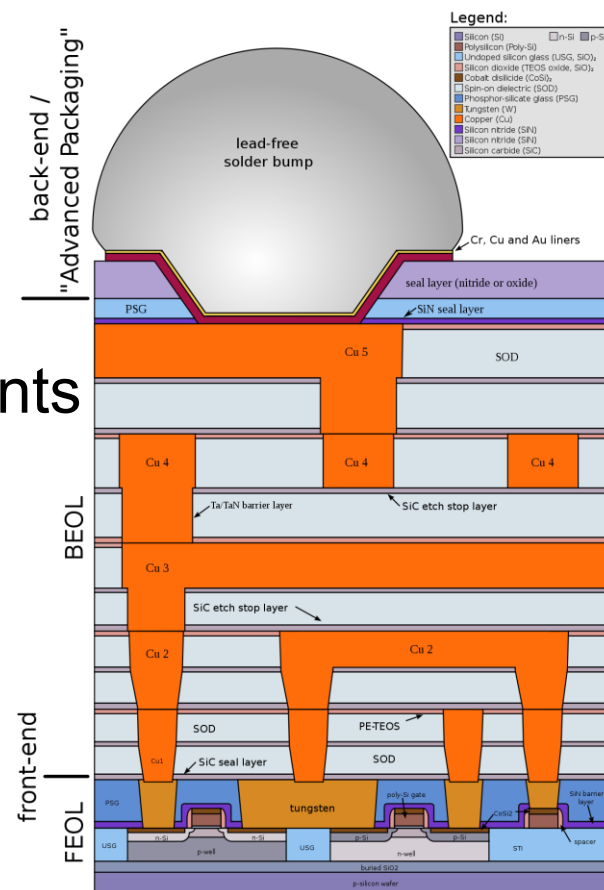
60 masks -> 16nm

77 masks -> 11nm

2D Layout for Chip Fabrication



2D layout represents
3D structures





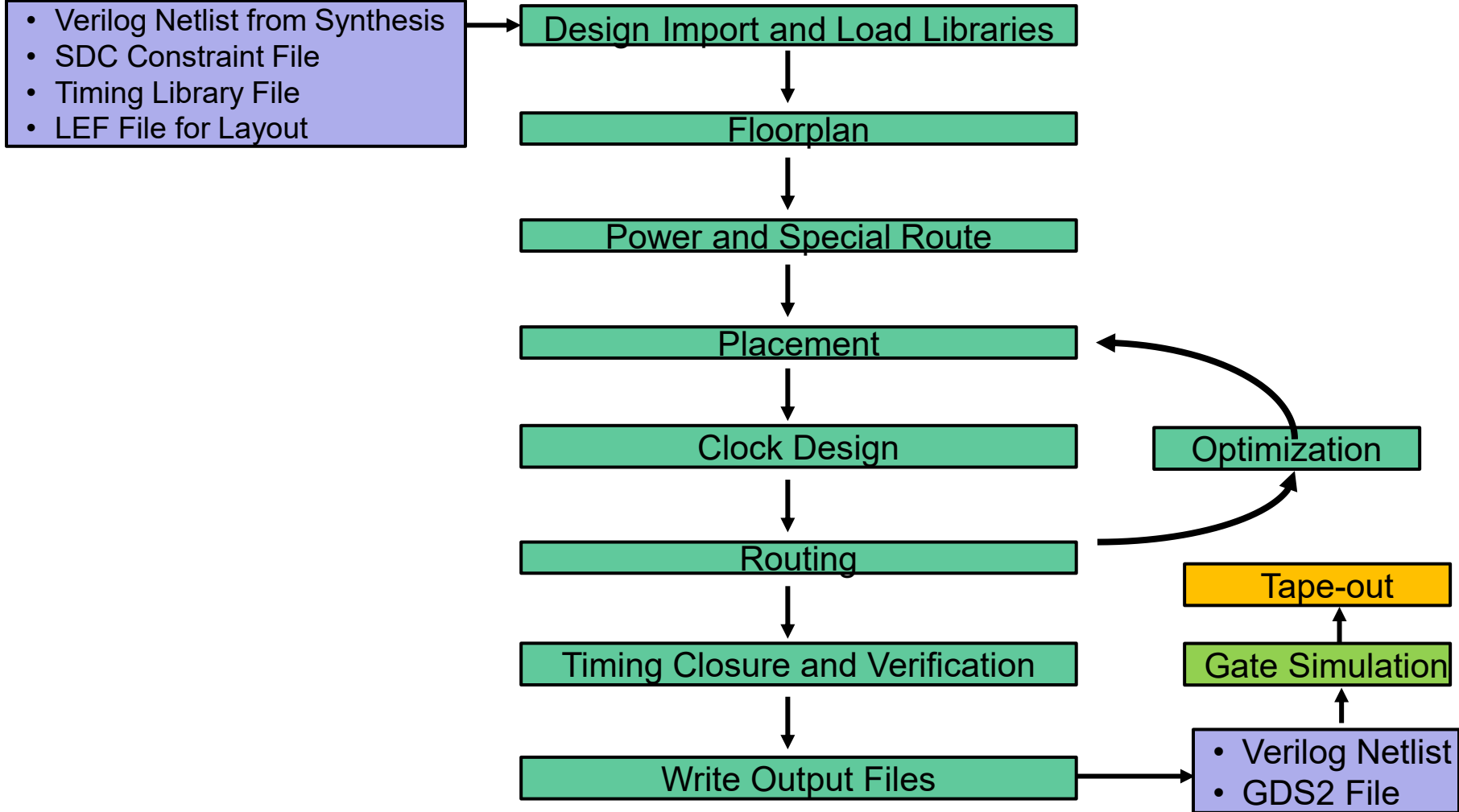
Chip Fabrication

<https://www.youtube.com/watch?v=qm67wbB5GmI>

<https://www.youtube.com/watch?v=aCOyq4YzBtY>



ASIC Backend Flow





Cadence Innovus

- Commonly used back-end design tool
- Support both GUI interface and script (*tcl format)
 - Command line with scripts are most common methods
 - But some steps are better to use GUI first for visual guide, then repeat/adjust with command line
 - All commands are saved in log and cmd files for later reuse
 - At every step, you can "saveDesign name" to a "*.enc" file so you can start from this step again
 - Each command can have hundreds of options and variables
 - It is always good to check help
 - Click on GUI help in the menu
 - Type "cdnshelp &"
 - Type "man <commandName>"
 - Type "<commandName> -help"



Innovus Instruction

- The following slides explain some important backend design steps in Innovus
- The commands are not complete; Please refer to Innovus tutorial for complete commands
- Practically, you should save your commands into a tcl file and source the file "source *.tcl" in Innovus to execute your commands in batch



Import Design

- Verilog netlist from synthesis
- LEF Files: layout of standard cells & Macros
- MMC View Definition: SDC file and corner definition
- Power nets: VDD, VSS

Note: you can “restore” your design at any stage by loading the saved “*enc” file



Import Design

Design Import

Netlist:

☒ Verilog

Files: ...

Top Cell: ☒ Auto Assign ☐ By User:

☐ OA

Library:

Cell:

View:

Technology/Physical Libraries:

☐ OA

Reference Libraries: ...

Abstract View Names:

Layout View Names:

☒ LEF Files

...

Floorplan

IO Assignment File: ...

Power

Power Nets:

Ground Nets:

CPF File: ...

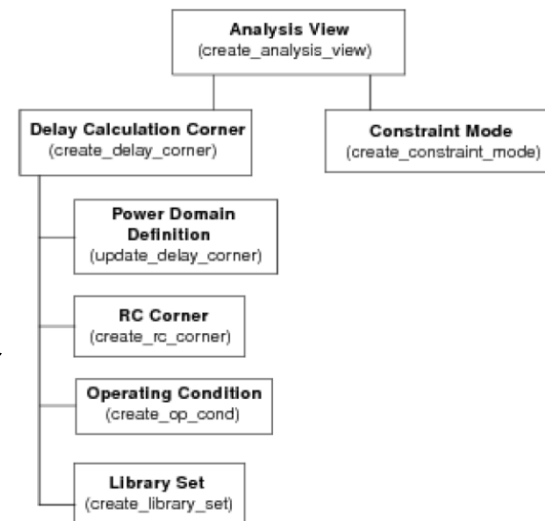
Analysis Configuration

MMM View Definition File: ...

Create Analysis Configuration ...

OK **Save...** **Load...** **Cancel** **Help**

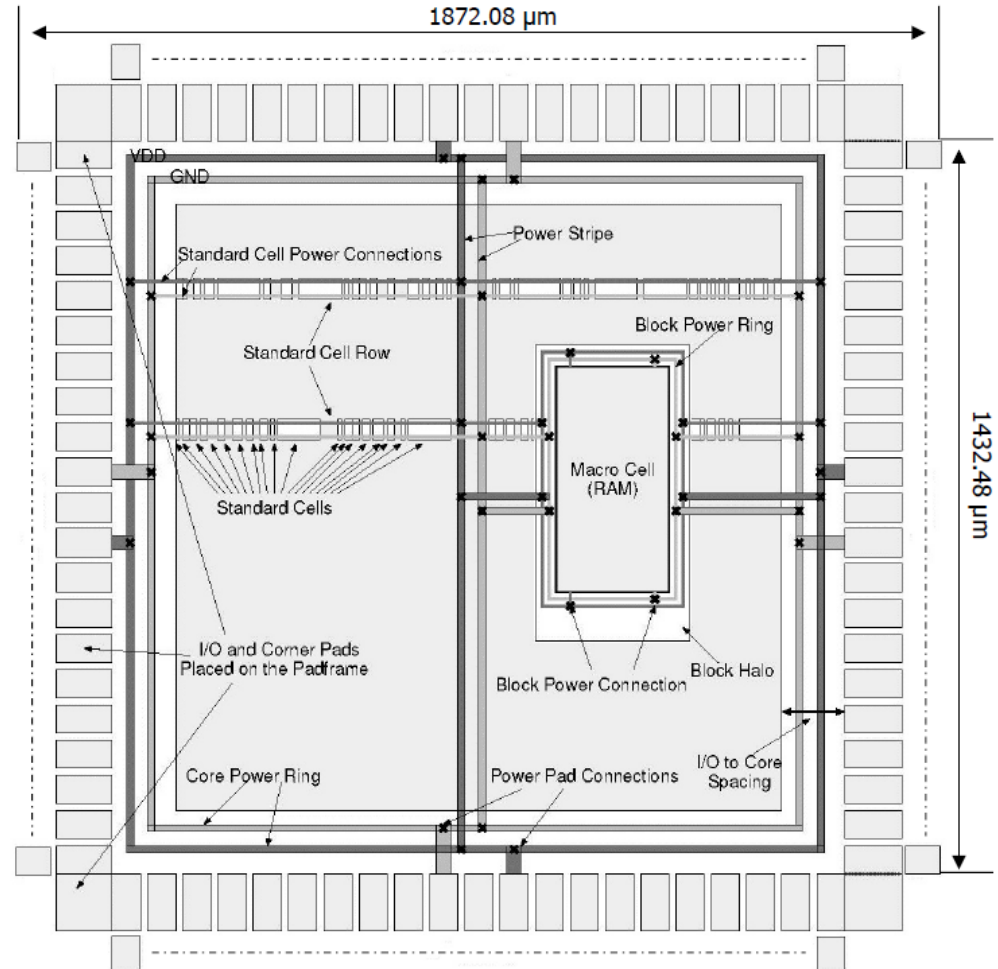
Multi-mode Multi-Corner (MMMC) defines PVT corners for timing closure and SDC file





Floorplanning

- Define chip size and aspect ratio
- Define IO ring
- Define power
- Define location of major macro, e.g. SRAM
- Define pin locations





Floorplanning

- A few important commands for floorplanning

set technology to 45nm

setDesignMode -process 45

aspect ratio of 1, 63% utilization area, 2um space at each side

another format is floorplan -d {W H Left Bottom Right Top} to fix dimension

floorPlan -r 1.0 0.63 2 2 2 2

set power supply to “VDD” and “VSS” pin of STD cells

globalNetConnect VDD -type pgpin -pin VDD -inst *

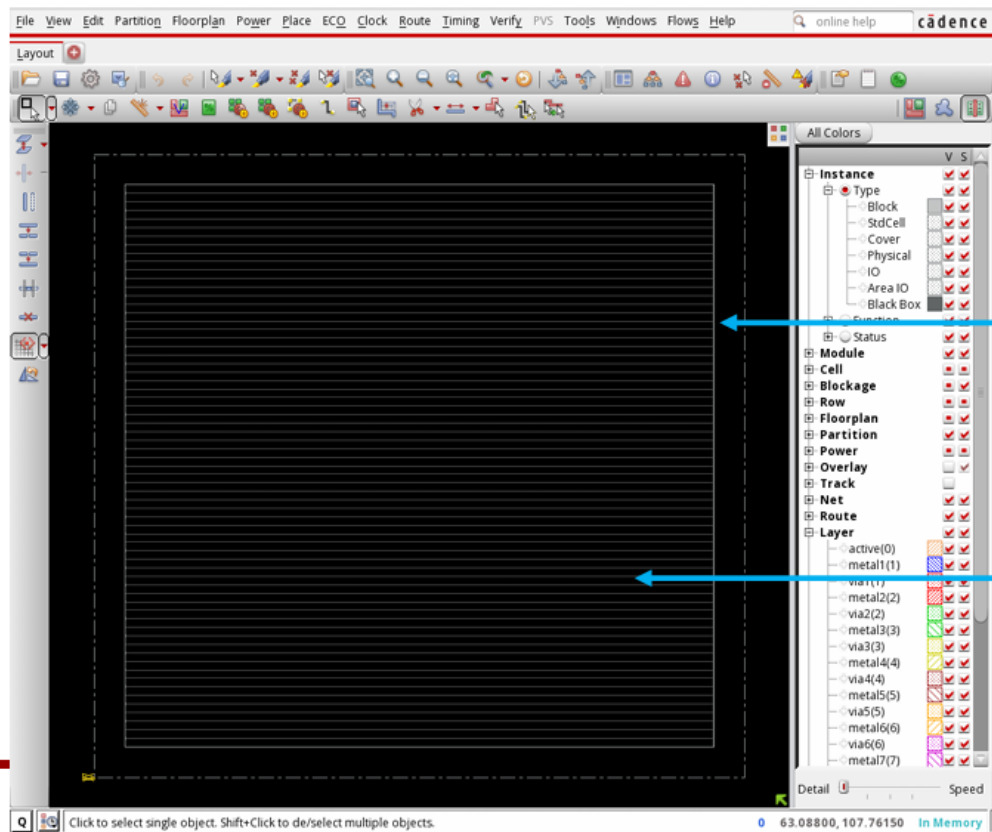
globalNetConnect VSS -type pgpin -pin VSS -inst *

Additional commands: setObjFPlanBox & createGuide to define location of Macro



Floorplanning

- You will see something as this



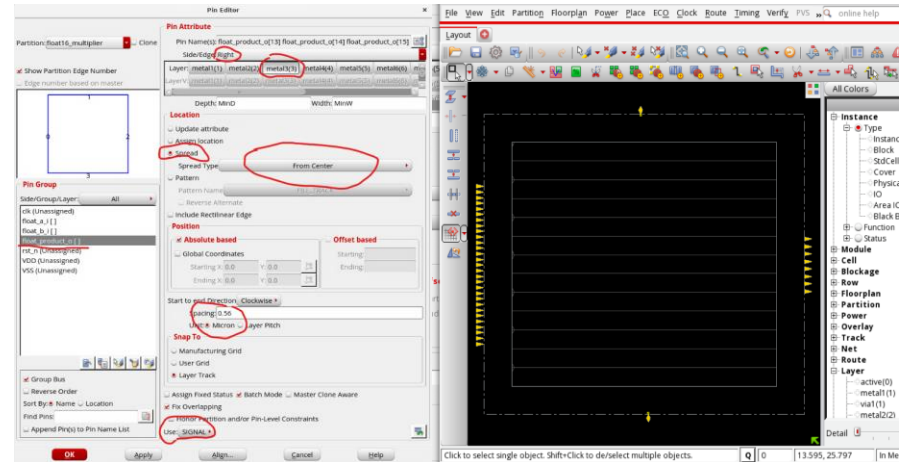
Power/Ground rings will be laid out in this area.

Std. cells will be placed in this core area.



Edit Pin

- Define input/output pin locations
- Easier to use GUI for first time
- Then use saved command line for refinement
- For example, put the output signals 'out[]'
 - Side/Edge: Right,
 - Layer: M3 layer,
 - Spread: from Center,
 - Spacing: 0.56.
 - Use: SIGNAL
- For VDD, VSS use Power and Ground pin





Power and Special Routing

Add power ring around your design

```
addRing -nets {VSS VDD} -type core_rings -follow io -layer {top metal5 bottom metal5 left metal4  
right metal4} -width {top 1 bottom 1 left 1 right 1} -spacing {top 1 bottom 1 left 1 right 1} -offset  
{top 0 bottom 0 left 0 right 0} -center 0 -extend_corner {} -threshold 0 -jog_distance 0 -  
snap_wire_center_to_grid None
```

Add power stripe across your design

```
$ addStripe -block_ring_top_layer_limit metal5 -max_same_layer_jog_length 1.6 -  
padcore_ring_bottom_layer_limit metal3 -set_to_set_distance 5 -stacked_via_top_layer metal10 -  
padcore_ring_top_layer_limit metal5 -spacing 1 -xleft_offset 1 -merge_stripes_value 0.095 -layer  
metal4 -block_ring_bottom_layer_limit metal3 -width 1 -nets {VSS VDD} -  
stacked_via_bottom_layer metal1
```

Make connections of power stripe using “via”

```
sroute -connect { blockPin padPin padRing corePin floatingStripe } -layerChangeRange { 1 10 } -  
blockPinTarget { nearestRingStripe nearestTarget } -padPinPortConnect { allPort oneGeom } -  
checkAlignedSecondaryPin 1 -blockPin useLef -allowJogging 1 -crossoverViaBottomLayer 1 -  
allowLayerChange 1 -targetViaTopLayer 10 -crossoverViaTopLayer 10 -targetViaBottomLayer 1 -  
nets { VDD VSS }
```



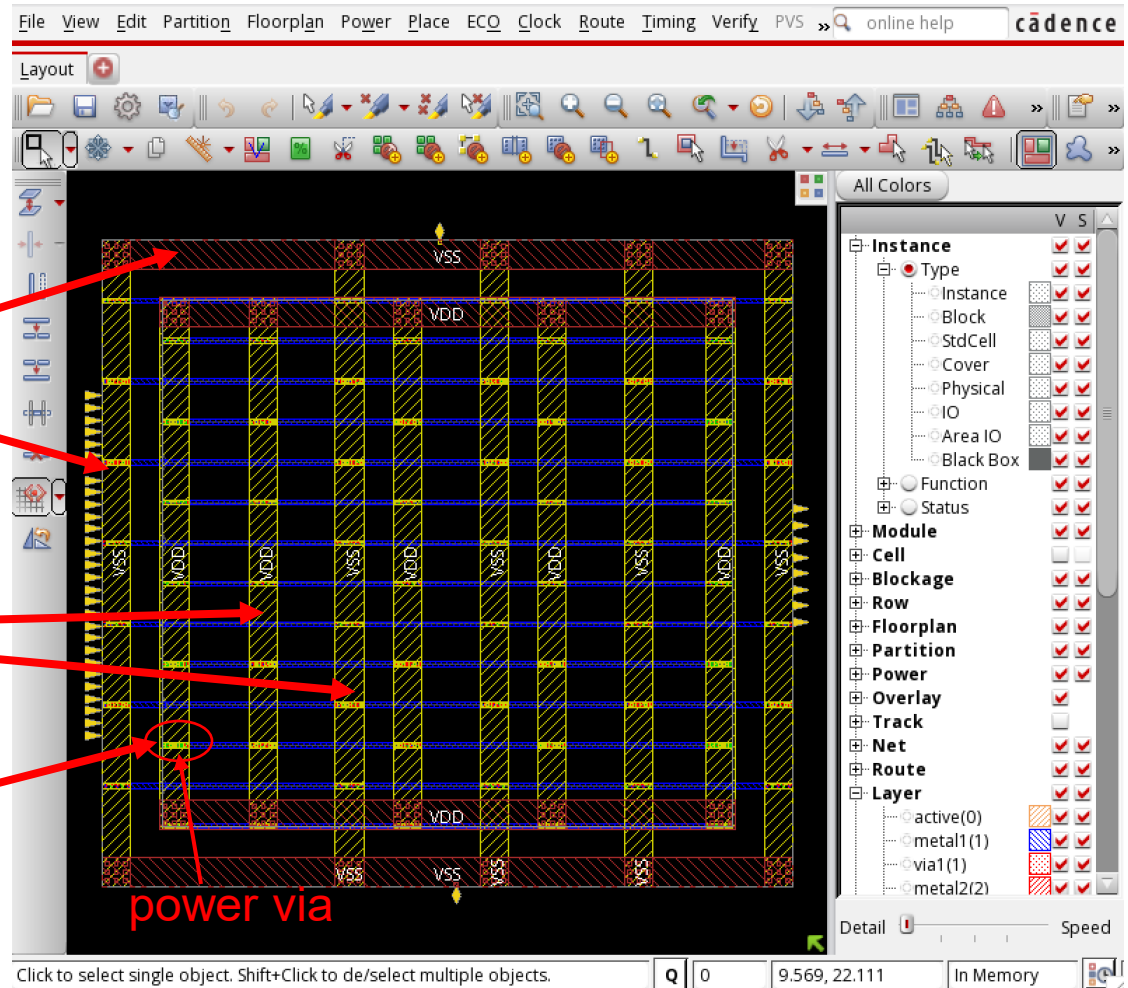

Power and Special Routing

addRing

addStripe

sroute

power via





Placement

- Placement places STD cells into the rows
- A preliminary timing analysis and optimization are performed
- Timing results are saved after "timeDesign"
- Special cells such as body tie cells, decap cells are also inserted at this stage
- Initial routing is also performed

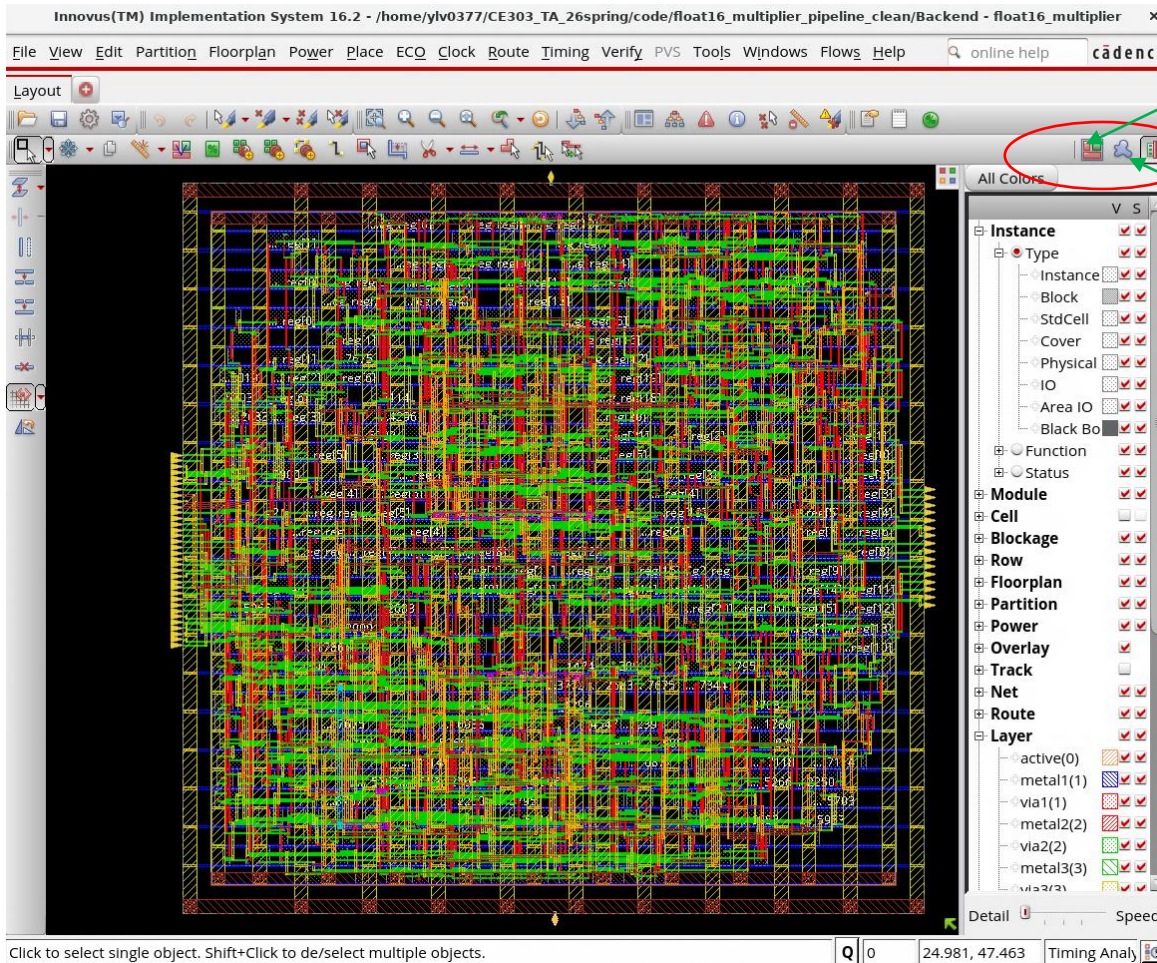
placeDesign

timeDesign -preCTS -numPaths 200

optDesign -preCTS -numPaths 200



Placement



Floorplan View (least detailed)

Physical View (most detailed)

Amoeba View

Click to select single object. Shift+Click to de/select multiple objects.

Q 0 24.981, 47.463 Timing Analy



Clock Design

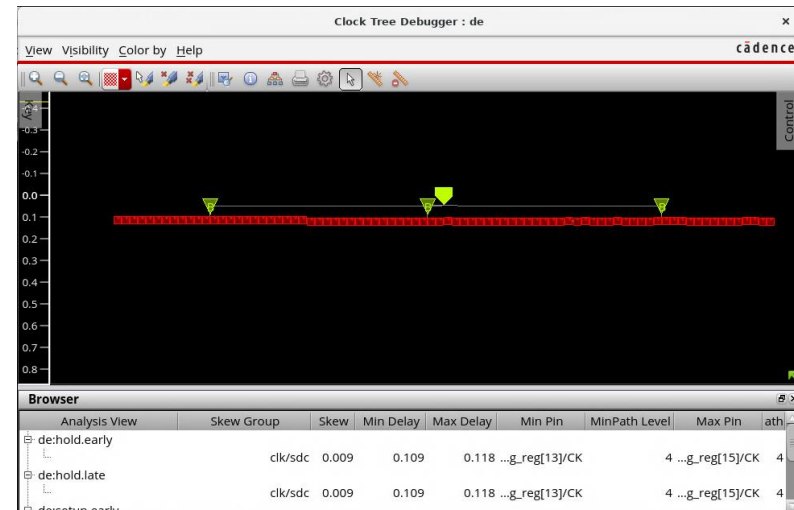
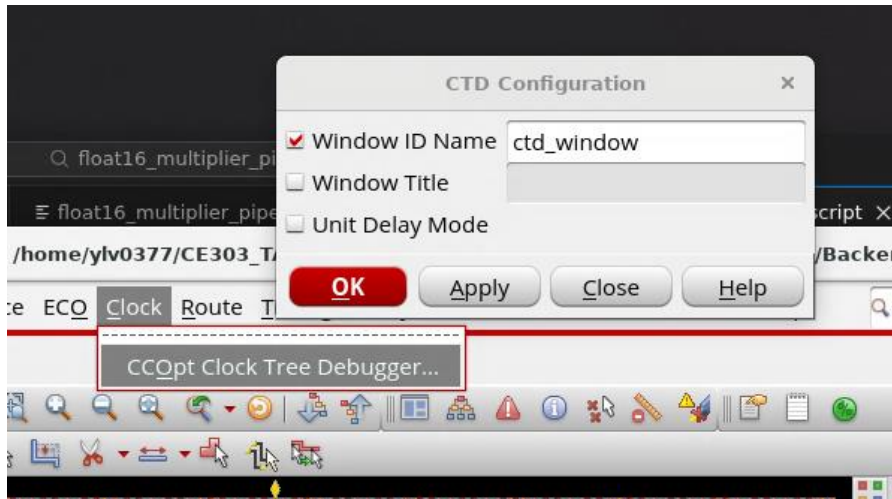
- Generate balanced clock tree (minimal clock skew)
- Perform timing analysis and optimization for both setup and hold (use optDesign to fix timing issues)
- Also perform initial routing
- Clock specification is saved in "clk_gen.ccopt" which can be modified; There are a lot of options
- This step can take a while if timing is hard to meet

```
create_ccopt_clock_tree_spec -file clk_gen.ccopt  
ccopt_design  
timeDesign -postCTS -numPaths 200  
timeDesign -postCTS -hold -numPaths 200  
optDesign -postCTS -numPaths 200  
optDesign -postCTS -hold -numPaths 200  
timeDesign -postCTS -hold -numPaths 200  
timeDesign -postCTS -numPaths 200
```



Clock Design

- open the Clock Browser from the Innovus GUI;
- You can visualize delay in clock tree which should have low clock skew at leaf





Detailed Routing

Fillers are added to fill in gaps in existing placement

```
addFillerGap 0.6
```

```
addFiller -cell FILLCELL_X1 FILLCELL_X2 FILLCELL_X4 FILLCELL_X8 -prefix  
FILLER -markFixed
```

Route the design; This step is the longest step in backend operation

```
routeDesign -globalDetail
```

```
timeDesign -postRoute -pathReports -drvReports -slackReports -numPaths 400 -prefix  
alu_conv_postRoute -outDir timingReports
```

```
timeDesign -postRoute -hold -pathReports -slackReports -numPaths 400 -prefix  
alu_conv_postRoute -outDir timingReports
```

```
optDesign -postCTS -numPaths 200
```

```
optDesign -postCTS -hold -numPaths 200
```

```
timeDesign -postCTS -hold -numPaths 200
```

```
timeDesign -postCTS -numPaths 200
```

Multiple rounds of optDesign may be performed to fix timing problems



Static Timing Analysis and Verification

- “timeDesign” performs STA
- All timing reports are saved under “timingReports” folder.
- You can find timing analysis for setup and hold under this folder.
- You can also type “report_timing” to get the timing results on the screen.
- All “slack” should be positive or nearly 0
- Also need to perform the following checks:

verify that there is no DRC violation

verify_drc -report cnn.drc.rpt -limit 1000

verify that there is no connection mistakes

verifyConnectivity -type all -error 1000 -warning 50

verify that there is no antenna violation

verifyProcessAntenna -reportfile cnn.antenna.rpt -
error 1000

```
*** Starting Verify DRC (MEM: 981.3) ***

VERIFY DRC ..... Starting Verification
VERIFY DRC ..... Initializing
VERIFY DRC ..... Deleting Existing Violations
VERIFY DRC ..... Creating Sub-Areas
VERIFY DRC ..... Using new threading
VERIFY DRC ..... Sub-Area : 1 of 1
VERIFY DRC ..... Sub-Area : 1 complete 0 Viols.

Verification Complete : 0 Viols.

*** End Verify DRC (CPU: 0:00:00.1 ELAPSED TIME: 1.00 MEM: 104.7M) ***

***** Start: VERIFY CONNECTIVITY *****
Start Time: Sun Sep 29 22:32:25 2019

Design Name: alu_conv
Database Units: 2000
Design Boundary: (0.0000, 0.0000) (23.9400, 22.4000)
Error Limit = 1000; Warning Limit = 50
Check all nets

Begin Summary
Found no problems or warnings.
End Summary

End Time: Sun Sep 29 22:32:25 2019
Time Elapsed: 0:00:00.0

***** End: VERIFY CONNECTIVITY *****
Verification Complete : 0 Viols. 0 Wrngs.
(CPU Time: 0:00:00.0 MEM: 0.000M)

***** START VERIFY ANTENNA *****
Report File: cnn.antenna.rpt
LEF Macro File: alu_conv.antenna.lef
Verification Complete: 0 Violations
***** DONE VERIFY ANTENNA *****
(CPU Time: 0:00:00.0 MEM: 0.000M)
```



Generate Output Files

#save netlist with all power connections and physical cells, e.g. fillers

```
saveNetlist -phys -includePowerGround float16_multiplier.v -excludeLeafCell
```

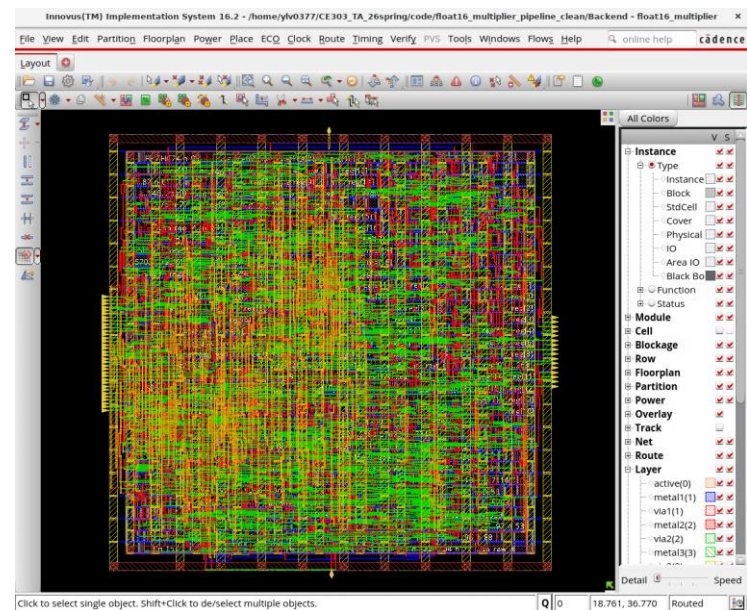
#save netlist without power and fillers for simulation purpose

```
saveNetlist float16_multiplier_nophy.v -excludeLeafCell
```

#write out timing file for gate-level simulation

in Standard Delay Format (SDF) file format

```
write_sdf float16_multiplier.sdf
```





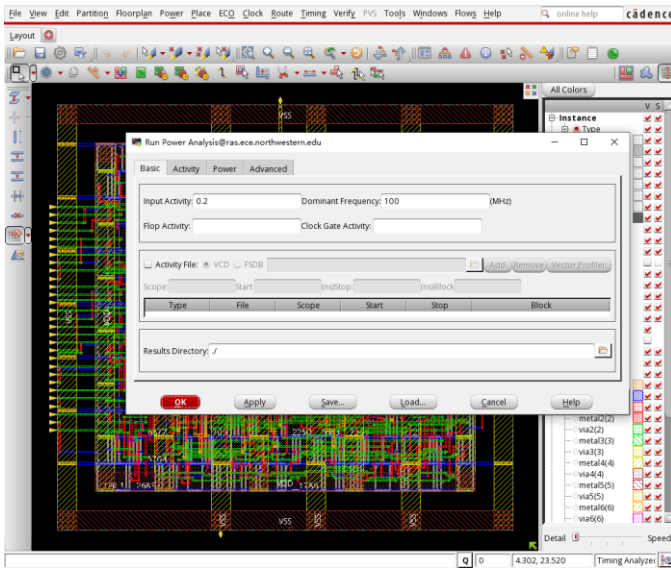
Other Reports

- You can report timing, area and power
 - “report_area”
 - “report_timing”
 - Report_timing (“report_timing -late” is for setup analysis, “report_timing -early” is for hold analysis; there are many options under report_timing)



Power Analysis

- To report power, you can go to Power -> Power analysis -> Run in Innovus.
- Set to the expected activity factor and clock frequency, then you can observe the design power in the terminal.
- More accurate way for power analysis is to use "Activity File" from Xcelium simulation and run dynamic power analysis w/ real waveforms



```
Total Power
-----
Total Internal Power:      0.01097859      35.0592%
Total Switching Power:     0.01436404      45.8703%
Total Leakage Power:       0.00597182      19.0705%
Total Power:               0.03131445
-----
Ended Static Power Report Generation: (cpu=0:00:00, real=0:00:00,
mem(process/total)=697.18MB/697.18MB)
```



Back-annotated Gate Simulation

- This is ultimate check to verify if the design works
- Delay (based on timing analysis) are annotated into the transition of each gate
- You can observe delay of each gate and simulate the functionality and timing of the design
- "SDF" file is used to annotate delay into each gate

```
xrun -64bit -gui -notimingcheck -f float16_multiplier_tb_gatesim.f -timescale  
1ns/10ps -access rwc -xmelab_args "-sdfstats temp -sdf_verbose -warnmax 0 -  
delay_mode none -mindelays" float16_multiplier_pipe_tb.v
```

- Alternatively, you can use zero delay mode "-delay_mode zero", but this does not check timing of the design

```
xrun -gui -access r -xmelab_args "-warnmax 0 -delay_mode zero -maxdelays"  
../Backend/float16_multiplier_nophy.v float16_multiplier_pipe_tb.v  
/vol/ece303/genus_tutorial/NangateOpenCellLibrary.v
```



Appendix